

BÀI THỰC HÀNH SỐ 3 VỀ MVC

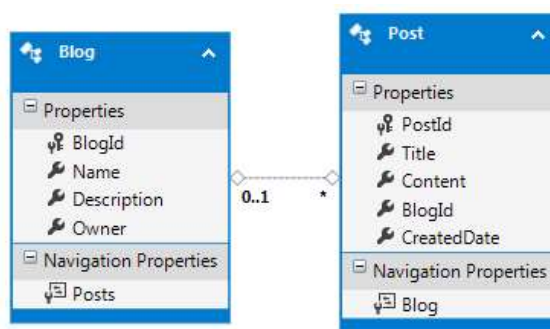
Nội dung:

- Cách sử dụng MetaClass để định nghĩa **DataAnnotations**
- Bắt lỗi dữ liệu đầu vào cơ bản ở MVC

Sử dụng dự án **MvcBlog** đã tạo ở Lab 3.

Câu 1. Thêm 1 số trường dữ liệu vào các bảng ở database như sau:

- Bảng Blog thêm trường Description và Owner với kiểu lần lượt là nvarchar(500) và nvarchar(50)
- Bảng Post thêm trường CreatedDate với kiểu datetime



Sau đó thử cập nhật lại database, mô hình Entity Framework và MVC bằng 1 trong 2 cách: xóa tạo lại hoặc thêm các trường còn thiếu vào View và Controller.

Câu 2. Khi cập nhật database (Database First) thì các định nghĩa cho các thuộc tính ở mô hình sẽ bị mất. Vì vậy để tránh trường hợp này thì nên thêm các lớp **metadata**.

- Ở thư mục Models thêm 1 tập tin mới đặt tên là **Metadata.cs**, trong đó chứa 2 lớp Postmetadata và Blogmetadata

```
using System.ComponentModel.DataAnnotations;
```

```
namespace MvcBlog.Models
```

```
{
```

```
    public class BlogMetadata
```

```
    {
```

```
        [StringLength(50)]
```

```
        [Display(Name = "Tên")]
```

```
        public string Name;
```

```
    }
```

```
    public class PostMetadata
```

```
    {
```

```
        // viết định nghĩa về tên tiếng Việt, độ dài dữ liệu, ... ở đây
```

```
    }
```

```
}
```

b. Viết toàn bộ định nghĩa về tên thuộc tính, chiều dài, ... ở bài **Lab 3** ở trong 2 lớp Postmetadata và Blogmetadata

c. Tiếp tục ở thư mục Models tạo thêm 1 tập tin tên là **PartialClasses.cs**. Để ý kỹ thấy ở 2 lớp Blog.cs và Class.cs dùng từ khóa **partial** để định nghĩa. Sinh viên tự tìm hiểu ý nghĩa từ khóa này. Tiếp theo, thêm nội dung ở PartialClasses.cs như sau:

```
using System.ComponentModel.DataAnnotations;
namespace MvcBlog.Models
{
    [MetadataType(typeof(BlogMetadata))]
    public partial class Blog
    {
    }

    [MetadataType(typeof(PostMetadata))]
    public partial class Post
    {
    }
}
```

Như vậy, đến bước này, khi cập nhập lại mô hình database thì các định nghĩa cho các thuộc tính để không bị mất nữa.

d. Thêm 1 trường dữ liệu tên là **Rank** ở bảng Blog với kiểu int. Sau đó cập nhật mô hình, View, Controller và xem kết quả.

Câu 3. Sử dụng các định nghĩa trong DataAnnotations, jQuery hoặc bất cứ phương pháp nào để bắt lỗi dữ liệu như sau:

a. Khi tạo mới và edit 1 Blog thì

- Tất cả các trường không được để trống
- Trường Name tối đa chỉ 50 ký tự
- Trường Description tối đa chỉ 500 ký tự
- Trường Owner tối đa chỉ 50 ký tự
- Trường Rank chỉ có giá trị từ 1 đến 100, chỉ nhập số không nhập được chữ
- Nếu Rank thì hiển thị vùng màu đỏ

b. Ở trang Index xem tất cả các Blog dựa vào Rank hiển thị vùng màu theo giá trị như sau:

- Nếu Rank từ 85 - 100 thì màu đỏ 
- Nếu Rank từ 70- 84 thì màu vàng 
- Nếu Rank từ 55 đến 69 thì màu xanh đậm (blue) 
- Nếu Rank từ 40 đến 54 thì màu xanh nhạt 
- Nếu Rank dưới 40 thì màu đen 

c. Khi tạo mới và edit 1 Post thì

- Trường Title, Content không được để trống dữ liệu
- Trường Title phải tối thiểu 5 ký tự và tối đa là 50 ký tự
- Trường Content phải tối thiểu 10 ký tự 500 ký tự
- Bắt buộc phải có BlogID, BlogID không thể bằng 0
- CreatedDate phải được nhập vào theo kiểu **dd/mm/yyyy**

d. Thử nhập đoạn mã <script> vào tất cả các trường dữ liệu text trong các Views xem điều gì xảy ra, sinh viên hãy bắt lỗi XSS (Cross-site scripting).